به نام خدا

آبان ماه ۱۴۰۵

تمرین دوم آزمون نرم افزار

استاد: دکتر احسان خامس‌پناه

«Mock ها Stub نیستند»

سلام 👋

امیدواریم خوب باشید،

با توجه به روز افزون شدن مبحث استفاده از Ai 🤖، به احتمال زیاد کد تست‌های این تمرین توسط شما با Ai نوشته می‌شوند که کاملا درست است و هیچ اشکالی ندارد ! 👏👏

اما از آنجایی که مبحث تست نرم‌افزار چه در تحقیقات و چه در بازار کار آینده به کمک شما خواهد آمد، نیاز است تا بدانید Ai برای شما چه تستی و با استفاده از چه تکنولوژی‌ای نوشته است و فایده و تاثیر یک تست چیست تا در آینده Ai نتواند از دانش کم شما در این مبحث سواستفاده کند 🙃 و کد و تستی با کیفیت پایین به شما تحویل دهد. (امروزه یکی از راه های اعتبارسنجی کد های نوشته شده توسط Ai دقیقا مبحث تست نرم‌افزار و تست‌نویسی صحیح است.)

بنابراین این تمرین بر این اساس طراحی شده است که شما گام به گام فایل صورت تمرین را خودتان می خوانید 👓 و بخش هایی که نیازمند یادگیری هستند را یاد گرفته ✍️ و در بخش هایی که نیاز به کد نویسی وجود دارد از Ai 🤖 استفاده می‌کنید.

نحوه‌ی نمره‌دهی 🧱 و تحویل‌گیری 🫳 تمرین هم به این صورت خواهد بود: کد نوشته‌شده توسط شما و Ai بخش کوچکی از نمره‌ی تمرین را تشکیل خواهد داد و اکثر نمره در یک تحویل مجازی بر اساس سوالاتی که دستیاران از مفهوم تست‌های نوشته شده و نحوه ی کارکرد تست‌ها خواهند پرسید، مشخص می‌شود.

هدف این تمرین آشنایی با بدل های آزمایشی (Test Doubles) است که برای بررسی و صحت سنجی نحوه‌ی تعامل برنامه با سایر قطعاتی است که از آن‌ها استفاده می‌کند. جهت نوشتن تست‌ها از Mockito که framework زبان جاوا برای ساخت بدل‌ها و استفاده از آن‌ها در آزمون نرم‌افزار است استفاده می‌کنید. بعد از مطالعه‌ی این صورت تمرین اگر هر سوال و ابهامی بود می‌توانید از ما[1] بپرسید.

موفق باشید 💪

مجید، محمد مهدی

---

[1] مجید صادقی نژاد (t.me/Msns42) و محمد مهدی صمدی (t.me/M1M7S)

Mock, Stub و سایرین 🤡

در یک برنامه که از نظر تست‌پذیری به خوبی طراحی شده باشد شما باید بتوانید اجزای داخلی برنامه (در برنامه نویسی Object-Oriented، همان Class ها) را به راحتی جایگزین کنید و آن‌ها را خارج از منطق برنامه به عنوان آرگومان به برنامه بدهید. حالا تصور کنید که می‌خواهید رفتار برنامه شبیه‌ساز یک هواپیما را بررسی کنید و ببینید اگر از خارج برنامه یک Object موتور هواپیما را، به عنوان آرگومان، به برنامه بدهید:

1. آیا برنامه در داخل خود، واقعا موتور هواپیما را روشن می‌کند؟
2. اگر موتور به برنامه بگوید که من خراب هستم (!) آیا برنامه واقعا پرواز را متوقف می‌کند؟

برای سوال اول شما نیاز به یک موتور هواپیمای بدلی دارید که آن را به عنوان آرگومان و ورودی به برنامه‌ی خود بدهید و پس از اجرای برنامه‌ی خود از این بدل سوال کنید که آیا واقعا method روشن شدن تو، هنگامی که داخل برنامه بودی، صدا شد؟ یعنی شما از Object بدلی خود، سوال می‌پرسید و رفتار انجام شده بر روی آن را صحت‌سنجی می‌کنید. نوع این بدل، Mock است چون شما از آن سوال می‌پرسید. و در حالت دوم شما یک موتور بدلی را به عنوان ورودی به سیستم می‌دهید اما پیش از اینکه موتور را به سیستم بدهید method بررسی خراب بودن موتور را به نحوی دستکاری می‌کنید که موتور بگوید من خراب هستم. یعنی رفتار قطعه‌ی ورودی را به نحوی تنظیم و دستکاری می‌کنید که مطابق شرایط مد نظر شما برای آزمون باشد. نوع این بدل Stub است. برای بررسی عمیق‌تر مباحث اشاره شده این مطلب را از سایت آقای Fowler بخوانید یا حتی ساده‌تر از آن، به دوست خود (Ai) مراجعه کنید.

Mockito و دوستان 🍹

در بخش قبل دیدیم که به چه منظوری نیاز به استفاده از بدل‌ها داریم و دو استفاده از بدل‌ها را بررسی کردیم. حال اگر به ازای هر بدل بخواهید یک Class جدید طراحی کنید که دقیقا رفتارهای مشابه Class واقعی را دارد، اما در یک رفتار به خصوص با Class واقعی متفاوت است، مقدار بسیار زیادی کد تکراری تولید می کنید و زمان بسیار زیادی را نیز تلف می‌کنید. همچنین اگر بخواهید Class خود را به نحوی طراحی کنید که بتوانید بعدا از آن در مورد رفتاری که با آن انجام شده سوال بپرسید، مقدار بسیار زیادی پیچیدگی غیر ضروری به Class خود اضافه می‌کنید. در همین راستا Framework یا کتابخانه‌ی Mockito در جاوا -که به کمک Maven و آموخته‌های تمرین پیشین می‌توانید از آن استفاده کنید- به کمک شما می‌آید. با Mockito می‌توانید Class واقعی و صحیحی که از قبل به عنوان قطعه‌ی داخلی سیستم خود استفاده کرده‌اید را به اصطلاح Mock کنید و یک Object تحویل بگیرید که دقیقا Field ها و Method هایی مشابه Class شما دارد و تایپی دقیقا مشابه Class شما دارد با این تفاوت که رفتار تک تک Method ها را می‌توانید خودتان مشخص کنید (Stub) و می‌توانید بعدا از این Object در مورد رفتاری که با آن صورت گرفته است به راحتی سوال بپرسید (Mock). با توجه به خواسته‌های تمرین، برای مشاهده‌ی نحوه‌ی کار و یادگیری اسم Function ها و Method های این کتابخانه می توانید از دوست خود (Ai) سوال کنید.

تست، تست، تست... ✈️

Code-Base مربوط به یک **شبیه‌ساز هواپیما** در اختیار شما قرار گرفته است و خواسته شده برای این برنامه تعدادی تست بنویسید تا از صحت عملکرد آن مطمئن شوید. در همین راستا لازم است تا بنا بر صلاحدید خودتان و نحوه‌ی طراحی برنامه، حداقل **۶** تست برای این برنامه بنویسید که در آن‌ها از مفاهیم زیر در کتابخانه ی Mockito استفاده شده است. (حداقل یکبار از هر مفهوم باید در مجموعه ی تست های شما استفاده شود. لازم نیست تمام مفاهیم در تک تک تست های شما به چشم بخورد.)

- ☐ **mock()**
- ☐ **reset()**
- ☐ **when()**
- ☐ **thenReturn()**
- ☐ **verify()**
- ☐ **times()**
- ☐ **any()**
- ☐ **thenThrow()**
- ☐ **doNothing()**

توضیحات

- تمرین های درس در گروه های دو نفره انجام می شوند.
- Code Base این پروژه در این مخزن Github موجود است. کدها را در یک مخزن خصوصی در Github بارگذاری کنید یا پروژه را fork کنید و کد تست‌های خود را به آن اضافه کنید.
- تا پایان تاریخ **(۱۴۰۵/۰۸/۲۳)** مخزن را خود را به صورت Private نگه دارید و پس از آن مخزن خود را به صورت Public دربیاورید تا دستیاران کد شما را بررسی کنند.
- در ایلرن آدرس مخزن Github تمرین خود را به همراه Hash آخرین کامیت (که تاریخ آن باید پیش از موعد تحویل باشد) را بارگذاری کنید (بارگذاری توسط یکی از اعضای تیم کافیست).
- این تمرین برای آشنایی بیشتر و یادگیری عملی مطالب درس طراحی شده است. در صورت پیدا شدن تقلب، مطابق قوانین درس برخورد خواهد شد.